

Concurrent Prime Number Calculation

Report & Analysis

For this assignment, I created a prime number calculator in both C++ and C#. The C++ version uses `std::thread`, while the C# version uses `Task.Run()` with `async/await`. Both programs calculate the number of primes between 2 and 10000000, with the work divided between 1, 2, 4, and 8 concurrent units.

The prime-checking algorithm was kept as similar as possible between both versions so the comparison focused mainly on the concurrency methods.

Code Structure

I opted to use a C++ version due to familiarity and the ability of C++ to give more direct control over the threads. I create the threads, divide the range between them, and use `join()` to make sure everything finishes before the program continues.

The C# version is a little cleaner because `Task.Run()` and `Task.WhenAll()` handle more of the task management. Instead of manually managing threads, I create tasks and wait for them to complete.

For this assignment, I found C++ more useful for understanding how threading actually works, while C# was easier to write and manage and maybe more useful in perhaps Unity development.

Performance in C++

1 Thread:

```
Prime Calculator
=====

Enter the upper limit: 10000000
Enter the number of threads: 1

System information
=====
Hardware threads detected: 12
Range: 2 - 10000000
Requested threads: 1

Running benchmark...
Each configuration will run 3 times.

Run 1: 4430.19 ms
Run 2: 4415.93 ms
Run 3: 4435.19 ms

Results
=====
Primes found: 664579
Average time: 4427.10 ms
Best time: 4415.93 ms

Done. The CPU may now recover.
```

2 Threads:

```
Prime Calculator
=====

Enter the upper limit: 10000000
Enter the number of threads: 2

System information
=====
Hardware threads detected: 12
Range: 2 - 10000000
Requested threads: 2

Running benchmark...
Each configuration will run 3 times.

Run 1: 2901.55 ms
Run 2: 2872.71 ms
Run 3: 2865.17 ms

Results
=====
Primes found: 664579
Average time: 2879.81 ms
Best time: 2865.17 ms

Done. The CPU may now recover.
```

4 Threads:

```
Prime Calculator
=====

Enter the upper limit: 10000000
Enter the number of threads: 4

System information
=====
Hardware threads detected: 12
Range: 2 - 10000000
Requested threads: 4

Running benchmark...
Each configuration will run 3 times.

Run 1: 1642.86 ms
Run 2: 1600.82 ms
Run 3: 1629.21 ms

Results
=====
Primes found: 664579
Average time: 1624.30 ms
Best time: 1600.82 ms

Done. The CPU may now recover.
```

8 Threads:

```
Prime Calculator
=====

Enter the upper limit: 10000000
Enter the number of threads: 8

System information
=====
Hardware threads detected: 12
Range: 2 - 10000000
Requested threads: 8

Running benchmark...
Each configuration will run 3 times.

Run 1: 904.90 ms
Run 2: 900.36 ms
Run 3: 909.01 ms

Results
=====
Primes found: 664579
Average time: 904.76 ms
Best time: 900.36 ms

Done. The CPU may now recover.
```

12 Threads:

```
Prime Calculator
=====

Enter the upper limit: 10000000
Enter the number of threads: 12

System information
=====
Hardware threads detected: 12
Range: 2 - 10000000
Requested threads: 12

Running benchmark...
Each configuration will run 3 times.

Run 1: 730.33 ms
Run 2: 724.50 ms
Run 3: 713.48 ms

Results
=====
Primes found: 664579
Average time: 722.77 ms
Best time: 713.48 ms

Done. The CPU may now recover.
```

Performance in C#

1 Task

```
Prime Calculator using C#
=====
Enter the upper limit: 10000000
Enter the number of tasks: 1

Results
-----
Range: 2 - 10000000
Tasks used: 1
Primes found: 664579
Time taken: 4885.22 ms
```

2 Tasks

```
Prime Calculator using C#
=====
Enter the upper limit: 10000000
Enter the number of tasks: 2

Results
-----
Range: 2 - 10000000
Tasks used: 2
Primes found: 664579
Time taken: 3150.21 ms
```

4 Tasks

```
Prime Calculator using C#
=====
Enter the upper limit: 10000000
Enter the number of tasks: 4

Results
-----
Range: 2 - 10000000
Tasks used: 4
Primes found: 664579
Time taken: 1793.61 ms
```

8 Tasks

```
Prime Calculator using C#
=====
Enter the upper limit: 10000000
Enter the number of tasks: 8

Results
-----
Range: 2 - 10000000
Tasks used: 8
Primes found: 664579
Time taken: 1021.42 ms
```

12 Tasks

```
Prime Calculator using C#
=====
Enter the upper limit: 10000000
Enter the number of tasks: 12

Results
-----
Range: 2 - 10000000
Tasks used: 12
Primes found: 664579
Time taken: 781.10 ms
```

The main pattern I observed was that increasing the number of workers improved performance at first, but the gains became smaller as more threads were added.

This makes sense because the CPU can only do so much work at once, and additional threads introduce scheduling overhead. In other words, throwing 8 threads at the problem does not automatically make it twice as fast as 4, it IS still faster, however.

I used a `std::mutex` to protect the shared prime count. Each thread calculates a local result first and only locks the mutex when adding that result to the total.

The C# version can avoid as much shared state by having each task return its own count and combining the results after `Task.WhenAll()` completes.

This was one of the most important parts of the assignment because without synchronization, multiple workers could update the same value at the same time and produce an incorrect result.

Challenges & Observations

The biggest challenge was dividing the range correctly without skipping or duplicating numbers.

Another interesting part was seeing that more concurrency does not always equal better performance. The benchmark made it much clearer where additional threads stopped being useful.

Overall, the C++ version gave me a better understanding of manual thread management, while the C# version showed how tasks can provide a simpler abstraction for concurrent work.

Both approaches successfully solved the problem, but they provide different levels of control. `std::thread` is more explicit and gives the programmer greater control, while `async/await` and the Task Parallel Library simplify task management.

The biggest takeaway for me was that good concurrency is not about using as many threads as possible. It is about dividing the work efficiently while keeping synchronization and overhead under control.